

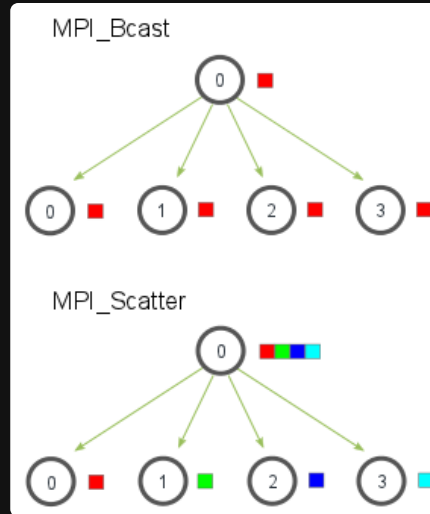
Collective Communication

Scatter

is a collective routine that is very similar to `Bcast`

It involves a designated root process and, sending data to all processes in a communicator

With the main difference being that scatter sends **chunks of an array** to different processes



Where the array of elements is distributed in order of rank

Scatter

```
1  MPC_Scatter(  
2      void* send_data,  
3      int send_count,  
4      MPI_Datatype send_datatype,  
5  
6      void* recv_data,  
7      int recv_count,  
8      MPI_datatype recv_datatype,  
9  
10     int root,  
11     MPI_Comm comm  
12 )
```

Scatter, send data

Assuming 4 cores

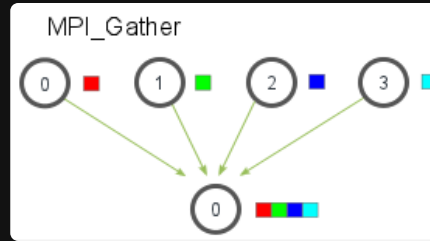
```
1  int recv_data[2];
2  int send_data[8] = {1, 2, 3, 4, 5, 6, 7, 8};
3
4  MPI_Scatter(
5      send_data,
6      2,
7      MPI_INT,
8
9      recv_data,
10     2,
11     MPI_INT,
12
13     2,
14     MPI_COMM_WORLD
15 );
```

Gather

is the *opposite* of scatter, where each process *sends data* to a designated root process

Where the root process receives the data in order of rank

Mostly used for parallel sorting and searching



Gather

```
1  MPI_Gather(  
2      void* send_data,  
3      int send_count,  
4      MPI_Datatype send_datatype,  
5  
6      void* recv_data,  
7      int recv_count,  
8      MPI_datatype recv_datatype,  
9  
10     int root,  
11     MPI_Comm comm  
12 )
```

*only the root needs a valid receive buffer

* `recv_count` is the number of elements received from each process, not the total

Gather

Assuming 4 cores

```
1  int send_data[2] = {1, 2};
2  int recv_data[8];
3
4  MPI_Gather(
5      send_data,
6      2,
7      MPI_INT,
8
9      recv_data,
10     2,
11     MPI_INT,
12
13     2,
14     MPI_COMM_WORLD
15 );
```

Computing the average of numbers

What is average of the list

```
1  int items[4] = {3,4,5,6};
```

Defining root

So given random numbers on the root

```
1  float *rand_nums;  
2  if (world_rank == 0) {  
3      rand_num = create_rand_nums(elements_per_proc * proc_count);  
4  }
```

Local variables and sending

Define a local variable to hold the average of each process

```
1  float *local_rand_nums = malloc(sizeof(float) * elements_per_proc);
2
3  MPI_Scatter(
4      rand_nums,
5      elements_per_proc,
6      MPI_FLOAT,
7
8      local_rand_nums,
9      elements_per_proc,
10     MPI_FLOAT,
11
12     0,
13     MPI_COMM_WORLD
14 );
```

Compute the average of the local numbers

```
1  float compute_avg(float* nums, int count) {
2      float sum = 0.0;
3      for (int i = 0; i < count; i++) {
4          sum += nums[i];
5      }
6      return sum / count;
7  }
8
9  float local_avg = compute_avg(local_rand_nums, elements_per_proc);
```

Gather the local averages

```
1  float *local_avg_array = NULL;
2  if (world_rank == 0) {
3      local_avg_array = malloc(sizeof(float) * proc_count);
4  }
5
6  MPI_Gather(
7      &local_avg,
8      1,
9      MPI_FLOAT,
10     local_avg_array,
11     1,
12     MPI_FLOAT,
13     0,
14     MPI_COMM_WORLD
15 );
```

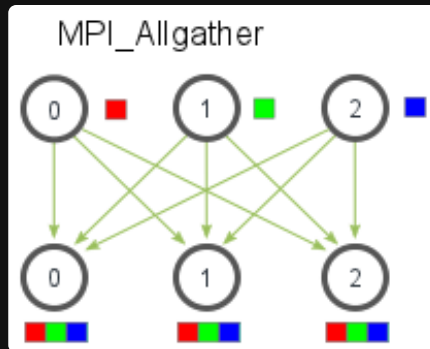
Compute the global average

```
1  if (world_rank == 0) {  
2      float global_avg = compute_avg(local_avg_array, proc_count);  
3      printf("Global average: %f\n", global_avg);  
4  }
```

Allgather

If scatter is a *one-to-many* operation and gather is a *many-to-one* operation, then allgather is a *many-to-many* operation

Where *each process* **sends** data to *all other processes*, and **receives** data from *all other processes*



In the most basic sense, it's an `MPI_Gather` followed by an `MPI_Bcast`

Allgather

The function declaration is similar to `MPI_Gather` but without the `root` argument

```
1  MPI_Allgather(  
2      void* send_data,  
3      int send_count,  
4      MPI_Datatype send_datatype,  
5  
6      void* recv_data,  
7      int recv_count,  
8      MPI_datatype recv_datatype,  
9  
10     MPI_Comm comm  
11 )
```

Modifying the average example

```
1  float *local_avg_array = malloc(sizeof(float) * proc_count);
2  MPI_Allgather(
3      &local_avg,
4      1,
5      MPI_FLOAT,
6
7      local_avg_array,
8      1,
9      MPI_FLOAT,
10
11     MPI_COMM_WORLD
12 );
13
14 float global_avg = compute_avg(local_avg_array, proc_count);
```

Where the only difference is that *all processes* receive the local averages, and *all processes* compute the global average

Semi final project 1

Scenario:

You are writing a program to process a very small, 16-pixel grayscale image, which is represented by a 1D array of 16 integers

Where each integer is a brightness value between 0 (black) and 255 (white)

Goal:

Apply a *brightness filter* to the image

Where every pixel needs its value increased by 50 but capped at 255 (so if the value is above 255, it should be set to 255)

Semi final project 1

Constraints:

1. it must run 4 processes
2. Process `0` should hold the image array
3. You must `scatter` the pixels equally among the 4 processes
4. Each process applies the brightness filter to its local chunk of pixels,
5. You must `gather` the processed pixels back to process `0`

Starter template can be found on NEO